

Clustering and PageRank

Instructions:

- Submit a report consisting of your understanding, reasonings and outputs. It is recommended to highlight whatever assumptions you take while solving a problem.
 - Write all your code in a Jupyter notebook and push it to GitHub. In the report, attach the link to the GitHub repo. Google Colab link won't be accepted.
 - Submit your report (consisting the GitHub repo link) in the PDF format and name it `<roll_number>_CSL7110_Assignment4.pdf`
 - Ensure all code is properly commented and executed before submission. Plagiarism detected will result in zero marks.
 - **Max marks: 120**
-

Part 1: Clustering

40 Marks

For this part of the algorithm, you have to study two new algorithms called **Farthest First algorithm** for k-center clustering and **k-means++**.

Reading for Farthest First Algorithm: http://www.wikiwand.com/en/Farthest-first_traversal

Papers and reading for K-Means++:

- <https://theory.stanford.edu/~sergei/papers/kMeansPP-soda.pdf>
- <http://theory.stanford.edu/~sergei/slides/BATS-Means.pdf>

There are other tons of material available online in the form of slides, YouTube videos which you can access to get a better understanding of the two algorithms.

Dataset: included in the folder: Q1- UCI Spam clustering

For this assignment we will work on points in Euclidean space represented by vectors of numbers. The dataset is made of **4601 points with 58-dimensions** which are features available to an email spam detection system.

In Spark, the points can be represented as instances of the class

`org.apache.spark.mllib.linalg.Vector` and can be manipulated through static methods offered by the class `org.apache.spark.mllib.linalg.Vectors` (these classes are available

both for Java and Python). For example, method `Vectors.dense(x)` transforms an array `x` of double into an instance of class `Vector`, while method `Vectors.sqdist(a,b)` computes the squared L2-distance between two instances `a` and `b` of class `Vector`.

You must develop **4 functions** that are Spark runnable:

1. A function `readVectorsSeq(filename)` that, given in input the name (or path) `filename` of a text file containing the feature points (one point per line with features separated by comma, as in the file above) transforms it into a list of `Vector`.
2. A function `kcenter(P, k)` that receives in input a set of points `P` and an integer `k`, and returns the set `C` of `k` centers computed by the **Farthest-First Traversal** algorithm. You can represent both `P` and `C` as lists of `Vector`.
3. A function `kmeansPP(P, k)` that receives in input a set of points `P` and an integer `k`, and returns a set `C` of `k` centers computed with the **kmeans++** algorithm.
4. A function `kmeansObj(P, C)` that receives in input a set of points `P` and a set of centers `C`, and returns the **average squared distance** of a point of `P` from its closest center (i.e., the kmeans objective function for `P` with centers `C`, divided by the number of points of `P`).

Make sure that `kcenter(P,k)` and `kmeansPP(P,k)` run in time $O(|P| \times k)$

Finally, you must create a Python program, which receives in input a set `P` of features (provided as a text file as the above dataset), and 2 integers `k`, `k1`, with `k < k1`. The program incorporates the functions developed above and does the following:

1. Runs `kcenter(P, k)` printing its running time.
2. Runs `kmeansPP(P, k)` to obtain a set of `k` centers `C`, and then runs `kmeansObj(P, C)` printing the returned value.
3. Runs `kcenter(P, k1)` to obtain a set of `k1` centers `X`; then runs `kmeansPP(X, k)` to extract a set of `k` centers `C` from `X`, and finally runs `kmeansObj(P, C)` printing the returned value. Here the idea is to test whether `k1 > k` centers extracted with the `kcenter` primitive can provide a good coreset on which running `kmeans++`. Of course, the larger `k1` and the better the set of centers computed by `kmeansPP(X, k)`.

Part 2: Web-Search

40 Marks

In this part we will build the basic data structure underlying search engines: an **inverted index**. We will use this inverted index to answer some simple search queries.

An Inverted Index for a Set of Webpages

Suppose we are given a set of webpages W . For our purposes, each webpage $w \in W$ will be considered to be a sequence of words $w_1, w_2, \dots, w_k$. Another way of representing the webpage could be to maintain a list of words along with the position(s) of the words in the webpage. For example consider a web page with the following text:

Data structures is the study of structures for storing data.

This can be represented as:

```
{(data : 1, 10), (structures : 2, 7), (study : 5), (storing : 9)}
```

Note that the small connector words like "is", "the", "of", "for" have not been stored. Words like this are referred to as **stop words** and are generally removed since they are very frequent and normally contain no information about the content of the webpage.

This representation of the webpage is similar to the index we see at the back of many books which tell us the page numbers where certain important terms used in the book may be found. In mathematical notation we would say that given a webpage $w = w_1, w_2, \dots, w_k$, the index of w is:

```
{u : i1(u), ..., il(u)) : wij(u) = u, 1 ≤ j ≤ l}
```

An **index** is used to find the location of a particular string (word) in a specific document or webpage, but when we move to a collection of webpages, we need to first figure out which of the web pages contain the string. For this we store an **inverted index**.

Let us suppose we are given a collection of webpages C . For each page $p \in C$, let us denote by $W(p)$ the set of all words (excluding stop words) that occur in p . Note that:

$$W(C) = \bigcup_{p \in C} W(p)$$

is the set of all words in our collection.

An inverted index for C will contain an entry for each word $w \in W$. This entry will contain tuples of the form (p, k) to indicate that w occurs in the k th position of page $p \in C$. The inverted index of C is defined as:

```
Inv(C) = {(w : {(p, k) : p ∈ C, p[k] = w}) : w ∈ W(C)}
```

Example collection:

1. Data structures is the study of structures for storing data.

2. Structural engineers collect data about structures.

Inverted index:

```
{
  (data      : {(1,1), (1,10), (2,4)}),
  (structures: {(1,2), (1,7), (2,6)}),
  (study     : {(1,5)}),
  (storing   : {(1,9)}),
  (structural: {(2,1)}),
  (engineers : {(2,2)}),
  (collect   : {(2,3)})
}
```

The Web Search Problem

Given a collection of webpages C and a sequence of words $q_1 \dots q_k$, find the "most relevant" set of pages $p_1, p_2, \dots p_A$ that contain as many of $q_1 \dots q_k$ as possible and return them in the order of decreasing "relevance."

A Scoring Function for Search Term Relevance

One of the simplest scoring functions is **term frequency-inverse document frequency (TF-IDF)**. Given a word w and a webpage p , the relevance score is defined as:

$$\text{relevance}_w(p) = tf_w(p) \times idf_w(p)$$

Term Frequency: The total number of occurrences of word w in webpage p , denoted $f_w(p)$, normalized by the total number of words in webpage p , denoted $|W(p)|$:

$$tf_w(p) = \frac{f_w(p)}{|W(p)|}$$

Inverse Document Frequency: The logarithm of the total number of webpages N divided by the number of webpages $n_w(p)$ that contain the word w :

$$idf_w(p) = \log \frac{N}{n_w(p)}$$

The Assignment

You have been provided with a folder containing webpages and two files: `actions.txt` (which has the search queries) and `answers.txt` (which contains the answers to those queries).

Classes to Implement

MySet

- `addElement(element)` — Add element to the set.
- `union(otherSet: MySet)` — Return `MySet` representing the union of the current set and `otherSet`.
- `intersection(otherSet: MySet)` — Return `MySet` representing the intersection of the current set and `otherSet`.

Position — Represents a tuple `<page p, word position i>`

- `Position(p: PageEntry, wordIndex: int)` — Constructor.
- `getPageEntry()` — Return `PageEntry p`.
- `getWordIndex()` — Return `wordIndex`.

WordEntry — Stores the list of word indices where a string is present in document(s).

- `WordEntry(word: String)` — Constructor.
- `addPosition(position: Position)` — Add a position entry.
- `addPositions(positions: list)` — Add multiple position entries.
- `getAllPositionsForThisWord()` — Return a list of all position entries.
- `getTermFrequency(word: String)` — Return the term frequency of the word in a webpage.

PageIndex — Stores one word-entry for each unique word in the document.

- `addPositionForWord(str: String, p: Position)` — Add position `p` to the word-entry of `str`. If a word entry already exists, add `p` to it; otherwise create a new word-entry.
- `getWordEntries()` — Return a list of all word entries.

PageEntry — Stores information related to a webpage.

- `PageEntry(pageName: String)` — Constructor. Reads the file and creates the page index.
- `getPageIndex()` — Returns the page index of this webpage.

MyHashTable — Implements the hashtable used by `InvertedPageIndex`. Maps a word to its word-entry.

- `getHashIndex(str: String)` — Hash function that maps a string to the index of its word-entry.
- `addPositionsForWord(w: WordEntry)` — Adds an entry to the hashtable. If no word-entry exists, create one; if one exists, merge with the existing word-entry.

InvertedPageIndex

- `addPage(p: PageEntry)` — Add a new page entry to the inverted page index.
- `getPagesWhichContainWord(str: String)` — Return a set of page-entries of webpages which contain the word `str`.

SearchEngine

- `SearchEngine()` — Constructor. Creates an empty `InvertedPageIndex`.
- `performAction(actionMessage: String)` — Main stub method. Takes an action as a string.

Actions

Action	Description
<code>addPage x</code>	Add webpage <code>x</code> to the search engine database. The contents are stored in a file named <code>x</code> in the webpages folder.
<code>queryFindPagesWhichContainWord x</code>	Print the names of webpages which contain the word <code>x</code> , comma-separated. If not found, print "No webpage contains word <code>x</code> ".
<code>queryFindPositionsOfWordInAPage x y</code>	Print the word indices where word <code>x</code> is found in document <code>y</code> , comma-separated. If not found, print "Webpage <code>y</code> does not contain word <code>x</code> ". If the webpage is not in the database, print "No webpage <code>y</code> found".

Points to Note

- Convert each word to **lowercase**.
- Do **not** store connector words, but **do** consider them when calculating word indices.
Connector words: { `a`, `an`, `the`, `they`, `these`, `this`, `for`, `is`, `are`, `was`, `of`, `or`, `and`, `does`, `will`, `whose` }.
- Replace punctuation marks with a space. Punctuation: { `}` `[` `]` `<` `>` `=` `(` `)` `.` `,` `;` `'` `"` `?` `#` `!` `-` `:` }
- Treat plural and singular forms as the same: (`stack`, `stacks`), (`structure`, `structures`), (`application`, `applications`).
- We have given you a set of connector words, punctuation marks, and singular-plural entries. Consider these list of words as exhaustive. You do not need to make any more exceptions in your code.
- `actions.txt` contains some simple search queries.

- `answers.txt` contains the answers to `actions.txt` for you to check your code.
-

Part 3: PageRank on Spark

40 Marks

Dataset: <https://github.com/pnijhara/PySpark-PageRank/tree/main/graph>

In this assignment you will be implementing the **PageRank** algorithm on Spark. The graph has `n = 1000` nodes and `m = 8192` edges. 1000 of these edges form a directed cycle which ensures the graph is connected. The dataset may have multiple directed edges between a pair of nodes, which should be treated as a single edge. The first column in `whole.txt` is the source vertex and the second column is the destination.

PageRank Definition

Assume a directed graph $G = (V, E)$ has `n` nodes and `m` edges with all nodes having positive out-degrees. M is an `n × n` matrix defined as:

$$M_{i,j} = \begin{cases} \frac{1}{\deg(i)} & \text{if } (i \rightarrow j) \in E \\ 0 & \text{otherwise} \end{cases}$$

where `deg(i)` is the out-degree of node `i`. If there are multiple outgoing edges, treat them as a single edge.

By definition of PageRank, assuming `1 - β` as the teleport probability and denoting the PageRank vector by column vector `r`:

$$r = \frac{1 - \beta}{n} A + \beta M r$$

where A is the `n × 1` unit vector.

Iterative PageRank Algorithm

1. Initialize $r_0 = \frac{1}{n} A$
2. For `i` from 1 to `k` do: $r^{(i)} = \frac{1 - \beta}{n} A + \beta M r^{(i-1)}$

Experiment

Run for **40 iterations** with **β = 0.8** on Spark and obtain the PageRank vector `r`. Process matrix `M` as an RDD and report:

1. **Node IDs of top 5 nodes** with the highest scores.
2. **Bottom 5 nodes** with the lowest scores.

Before running on the whole graph, run the code on the smaller graph `small.txt` with **53 nodes**. The top-most score in this small graph is **0.036**.